

Теоретический материал

ПЕРЕМЕННЫЕ

Для хранения данных в программе применяются **переменные**.

Переменная представляет именнованную область памяти, в которой хранится значение определенного типа. Переменная имеет тип, имя и значение. Тип определяет, какого рода информацию может хранить переменная.

Перед использованием любую переменную надо определить. Синтаксис определения переменной выглядит следующим образом:

```
тип имя_переменной;  
int x;
```

ТИПЫ ДАННЫХ

В языке C# есть следующие базовые типы данных:

- **bool**: хранит значение true или false (логические литералы). Представлен системным типом **System.Boolean**
- **byte**: хранит целое число от 0 до 255 и занимает 1 байт. Представлен системным типом **System.Byte**
- **sbyte**: хранит целое число от -128 до 127 и занимает 1 байт. Представлен системным типом **System.SByte**
- **short**: хранит целое число от -32768 до 32767 и занимает 2 байта. Представлен системным типом **System.Int16**
- **ushort**: хранит целое число от 0 до 65535 и занимает 2 байта. Представлен системным типом **System.UInt16**
- **int**: хранит целое число от -2147483648 до 2147483647 и занимает 4 байта. Представлен системным типом **System.Int32**. Все целочисленные литералы по умолчанию представляют значения типа **int**:
- **uint**: хранит целое число от 0 до 4294967295 и занимает 4 байта. Представлен системным типом **System.UInt32**
- **long**: хранит целое число от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807 и занимает 8 байт. Представлен системным типом **System.Int64**
- **ulong**: хранит целое число от 0 до 18 446 744 073 709 551 615 и занимает 8 байт. Представлен системным типом **System.UInt64**
- **float**: хранит число с плавающей точкой от $-3.4 \cdot 10^{38}$ до $3.4 \cdot 10^{38}$ и

занимает 4 байта. Представлен системным типом **System.Single**

- **double**: хранит число с плавающей точкой от $\pm 5.0 \cdot 10^{-324}$ до $\pm 1.7 \cdot 10^{308}$ и занимает 8 байта. Представлен системным типом **System.Double**
- **decimal**: хранит десятичное дробное число. Если употребляется без десятичной запятой, имеет значение от $\pm 1.0 \cdot 10^{-28}$ до $\pm 7.9228 \cdot 10^{28}$, может хранить 28 знаков после запятой и занимает 16 байт. Представлен системным типом **System.Decimal**
- **char**: хранит одиночный символ в кодировке Unicode и занимает 2 байта. Представлен системным типом **System.Char**. Этому типу соответствуют символьные литералы:
- **string**: хранит набор символов Unicode. Представлен системным типом **System.String**. Этому типу соответствуют строковые литералы.
- **object**: может хранить значение любого типа данных и занимает 4 байта на 32-разрядной платформе и 8 байт на 64-разрядной платформе. Представлен системным типом **System.Object**, который является базовым для всех других типов и классов .NET.

КОНСОЛЬНЫЙ ВЫВОД

Для вывода информации на консоль мы уже использовали встроенный метод **Console.WriteLine**. То есть, если мы хотим вывести некоторую информацию на консоль, то нам надо передать ее в метод **Console.WriteLine**:

```
Console.WriteLine("Добро пожаловать в C#!");
```

Нередко возникает необходимость вывести на консоль в одной строке значения сразу нескольких переменных. В этом случае мы можем использовать прием, который называется **интерполяцией**:

```
1 string name = "Tom";
2 int age = 34;
3 double height = 1.7;
4 Console.WriteLine($"Имя: {name} Возраст: {age} Рост: {height}м");
```

Для встраивания отдельных значений в выводимую на консоль строку используются фигурные скобки, в которые заключается встраиваемое значение. Это можем значение переменной (`{name}`) или более сложное выражение (например, операция сложения `{4 + 7}`). А перед всей строкой ставится знак доллара `$`.

При выводе на консоль вместо помещенных в фигурные скобки выражений будут выводиться их значения:

Есть другой способ вывода на консоль сразу нескольких значений:

```
1 string name = "Tom";
```

```
2  int age = 34;
3  double height = 1.7;
4  Console.WriteLine("Имя: {0} Возраст: {2} Рост: {1}м", name, height, age);
```

КОНСОЛЬНЫЙ ВВОД

Кроме вывода информации на консоль мы можем получать информацию с консоли. Для этого предназначен метод **Console.ReadLine()**. Он позволяет получить введенную строку.

```
1  Console.Write("Введите свое имя: ");
2  string? name = Console.ReadLine();
3  Console.WriteLine($"Привет {name}");
```

В данном случае все, что вводит пользователь, с помощью метода **Console.ReadLine()** передается в переменную **name**.

Особенностью метода **Console.ReadLine()** является то, что он может считать информацию с консоли только в виде строки. Кроме того, возможная ситуация, когда для метода **Console.ReadLine** не окажется доступных для считывания строк, то есть когда ему нечего считывать, он возвращает значение **null**, то есть, грубо говоря, фактически отсутствие значения. И чтобы отразить эту ситуацию мы определяем переменную **name**, в которую получаем ввод с консоли, как переменную типа **string?**. Здесь **string** указывает, что переменная может хранить значения типа **string**, то есть строки. А знак вопроса **?** указывает, что переменная также может хранить значение **null**, то есть по сути не иметь никакого значения.

Однако, может возникнуть вопрос, как нам быть, если, допустим, мы хотим ввести возраст в переменную типа **int** или другую информацию в переменные типа **double** или **decimal**? По умолчанию платформа .NET предоставляет ряд методов, которые позволяют преобразовать различные значения к типам **int**, **double** и т.д. Некоторые из этих методов:

- **Convert.ToInt32()** (преобразует к типу **int**)
- **Convert.ToDouble()** (преобразует к типу **double**)
- **Convert.ToDecimal()** (преобразует к типу **decimal**)

Задание 1.1

Задача:

Написать программу реализующую функционал классического калькулятора средствами языка C#, предусмотреть реализацию следующих операций:

+, -, *, /, %, 1/x, x^2 , корень квадратный из x, M+, M-, MR.

В раздел решения приложить код решения и текстовое описание программного продукта по следующему плану:

1. Функционал;
2. Ограничения;
3. Возможные ошибки.

Решение:

```
using System;
```

```
class Calculator
```

```
{
```

```
    static void PrintLogo()
```

```
    {
```

```
        Console.WriteLine(
```

```
            ""
```

```
            $$$$$\ $$\ $$\ $$$$$\ $$$$$\ $$\ $$$$$\
```

```
            $$ _$$\ $$\ $$\ $$ _$$\ $$ _$$\ $$ _$$\
```

```
            $$ / \_ | $$$$$$$$$\ $$ / \_ | $$ / $$ | $$ | $$ / \_ |
```

```
            $$ | \_ $$ $$ | $$ | $$$$$$$$ | $$ | $$ |
```

```
            $$ | $$$$$$$$$\ $$ | $$ _$$ | $$ | $$ |
```

```
            $$ | $$\_ $$ $$ _ | $$ | $$\ $$ | $$ | $$ | $$\
```

```
            \$$$$$ | $$ | $$ | \$$$$$ | $$ | $$ | $$$$$$$$ \$$$$$ |
```

```
            \_ / \_ | \_ / \_ | \_ / \_ | \_ /
```

```
            ""
```

```
        );
```

```
    }
```

```
    static void PrintOperationInstructions(bool useMemory)
```

```
    {
```

```
        Console.WriteLine(
```

```
            $""
```

Введите число для выбора операции:

- (1) Сложить два числа (+)
- (2) Вычесть два числа (-)
- (3) Умножить два числа (*)
- (4) Разделить два числа (/)
- (5) Остаток от деления (%)
- (6) 1/x
- (7) Квадрат числа
- (8) Квадратный корень из числа

```

        (9) Сохранить число в память (M+)
        (10) Очистить память (M-)
        (11) Использовать число из памяти как первое входное число? (((useMemory) ?
"да" : "нет"))
        (12) Посмотреть память (MR)
        (13) Показать эту инструкцию снова
        (0) ВЫЙТИ ИЗ ПРОГРАММЫ

        ""
    );
}

static double GetValidatedNumber()
{
    const double maxAllowed = 1e12;
    const double minAllowed = -1e12;

    Console.WriteLine("Введите число: ");
    string input = Console.ReadLine().Replace('.', ',');

    if (!double.TryParse(input, out double number))
    {
        Console.WriteLine("Ошибка: введено не число!");
        return double.NaN;
    }

    if (number <= minAllowed || number >= maxAllowed)
    {
        Console.WriteLine("Ошибка: число вне допустимого диапазона!");
        return double.NaN;
    }

    return number;
}

public static void Main()
{
    double memory = 0;
    bool useMemory = false;
    PrintLogo();
    PrintOperationInstructions(useMemory);

    while (true)
    {
        Console.WriteLine("\nВведите операцию: ");
        string input = Console.ReadLine();

        if (!short.TryParse(input, out short operation))
        {
            Console.WriteLine("Неизвестная операция");
            continue;

```

```

    }

    double a, b, result;
    switch (operation)
    {
        case 0:
            return;

        case 1:
        case 2:
        case 3:
        case 4:
        case 5:
            a = useMemory ? memory : GetValidatedNumber();
            if (double.IsNaN(a)) break;
            b = GetValidatedNumber();
            if (double.IsNaN(b)) break;

            if (operation == 4 && b == 0)
            {
                Console.WriteLine("Ошибка: деление на ноль!");
                break;
            }

            if (operation == 5 && b == 0)
            {
                Console.WriteLine("Ошибка: деление на ноль!");
                break;
            }

            result = operation switch
            {
                1 => a + b,
                2 => a - b,
                3 => a * b,
                4 => a / b,
                5 => a % b,
                _ => 0
            };
            Console.WriteLine($"Итог: {result}");
            break;

        case 6:
        case 7:
        case 8:
            a = useMemory ? memory : GetValidatedNumber();
            if (double.IsNaN(a)) break;

            if (operation == 6 && a == 0)
            {
                Console.WriteLine("Ошибка: деление на ноль!");
            }
    }

```

```

        break;
    }

    if (operation == 8 && a < 0)
    {
        Console.WriteLine("Ошибка: квадратный корень из отрицательного
числа!");
        break;
    }

    result = operation switch
    {
        6 => 1.0 / a,
        7 => a * a,
        8 => Math.Sqrt(a),
        _ => 0
    };
    Console.WriteLine($"Итог: {result}");
    break;

case 9:
    a = GetValidatedNumber();
    if (!double.IsNaN(a)) memory = a;
    break;

case 10:
    memory = 0;
    Console.WriteLine("Память очищена");
    break;

case 11:
    useMemory = !useMemory;
    Console.WriteLine($"Использование памяти: {(useMemory ? "вкл" : "выкл")}");
    break;

case 12:
    Console.WriteLine($"Память: {memory}");
    break;

case 13:
    PrintOperationInstructions(useMemory);
    break;

default:
    Console.WriteLine("Неизвестная операция");
    break;
    }
}
}
}

```

Ответ:

```

$$$$$$\ $$\ $$\   $$$$$$\ $$$$$$\ $$\   $$$$$$\
$$ _$$\ $$\ $$\   $$ _$$\ $$ _$$\ $$\ |   $$ _$$\
$$ / \_||$$$$$$$$$$\   $$ / \_||$$ / $$|$$|   $$ / \_||
$$ |   \_$$ $$ $|   $$ |   $$$$$$$$|$$|   $$ |
$$ |   $$$$$$$$$$$\   $$ |   $$ _$$|$$|   $$ |
$$ | $$\ \_$$ $$ $|   $$ | $$\ $$|   $$|$$|   $$| $$\
\$$$$$$$| $$|$$|   \$$$$$$$|$$| $$|$$$$$$$$$\$$$$$$$|
\_||\_||   \_||\_||   \_||\_||   \_||\_||\_||\_||\_||\_||

```

Введите число для выбора операции:

- (1) Сложить два числа (+)
- (2) Вычесть два числа (-)
- (3) Умножить два числа (*)
- (4) Разделить два числа (/)
- (5) Остаток от деления (%)
- (6) 1/x
- (7) Квадрат числа
- (8) Квадратный корень из числа
- (9) Сохранить число в память (M+)
- (10) Очистить память (M-)
- (11) Использовать число из памяти как первое входное число? (нет)
- (12) Посмотреть память (MR)
- (13) Показать эту инструкцию снова
- (0) ВЫЙТИ ИЗ ПРОГРАММЫ

Введите операцию: 1
Введите число: 10
Введите число: 100
Итог: 110

Введите операцию: 9
Введите число: 100

Введите операцию: 11
Использование памяти: вкл

Введите операцию: 5
Введите число: 2
Итог: 0

Введите операцию: 2
Введите число: 10
Итог: 90

Введите операцию: 6
Итог: 0,01

Введите операцию: 8

Итог: 10

Введите операцию: 12

Память: 100

Введите операцию: 10

Память очищена

Введите операцию: 0